

★ Member-only story

A Step-by-Step Guide to Docker Image Optimisation: Reduce Size by Over 95%



Prateek Jain

14 min read · May 4, 2025



26



Open in app ↗



Medium



Search



Write



1.22 GB



57 MB

When I first deployed a Node.js app to production using Docker, I was stunned, The final image size was a whopping **1.2 GB**. All for just a few kilobytes of JavaScript code. It wasn't just about wasted storage. The oversized image slowed down deployments, hiked up costs, and increased our attack surface. After a few days of tinkering and trial-and-error in real projects, I found techniques that helped me shrink Docker images by **up to 95%**.

Friends link for non-Medium members: [A Step-by-Step Guide to Docker Image Optimisation: Reduce Size by Over 95%](#)

“Every megabyte matters when you're deploying thousands of containers.”

In this post, I'll take you through that journey, from bloated beginnings to lean, production-ready containers.

If you enjoy content like this, feel free to connect with me on X ([@PrateekJainDev](#)) and LinkedIn ([in/prateekjaindev](#)).

The Real Impact of Bloated Images

Image size directly affects how your applications behave in production. The difference between a 1GB image and a 50MB one can impact everything from startup time to infrastructure costs.

- **Faster Deployments:** Smaller images move through CI/CD pipelines faster. When you're shipping dozens of services daily, shaving a few minutes per deploy adds up quickly.
- **Reduced Costs:** Lean images consume less storage. Teams have seen up to 60% savings on container registry costs after trimming image bloat.

- **Better Security Posture:** Fewer packages mean fewer attack vectors. In one case, over two-thirds of flagged vulnerabilities were tied to unnecessary dependencies.

Docker image size directly impacts application startup time, which affects autoscaling, recovery time, and user experience.

Our Sample Application

Let's work with a typical Express application that represents the kind of Node.js microservice you might deploy in production:

It has essential dependencies like `express` for routing, `moment` for time formatting, `mongoose` for MongoDB interactions, and a couple of middlewares like `cors` and `helmet` to handle cross-origin requests and improve security. In a real-world microservice, you might also have logging, metrics, or authentication middleware, but for simplicity, we'll stick to the basics here.

First, initialise a new Node.js project:

```
mkdir docker-shrink
cd docker-shrink
npm init -y
```

Install the dependencies:

```
npm install express moment mongoose cors helmet  
npm install --save-dev nodemon jest supertest
```

Now, create an `index.js` file:

```
const express = require("express");  
const moment = require("moment");  
const cors = require("cors");  
const helmet = require("helmet");  
  
// Initialize express  
const app = express();  
const PORT = process.env.PORT || 3000;  
  
// Middleware  
app.use(cors());  
app.use(helmet());  
app.use(express.json());  
  
// Routes  
app.get("/", (req, res) => {  
  res.json({  
    message: "Hello from a slim Docker container!",  
    timestamp: moment().format("MMMM Do YYYY, h:mm:ss a"),  
    uptime: process.uptime(),  
  });  
});  
  
app.get("/health", (req, res) => {  
  res.json({ status: "UP", memory: process.memoryUsage() });  
});  
  
// Error handling middleware  
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).json({ error: "Something went wrong!" });  
});  
  
// Start server  
if (process.env.NODE_ENV !== "test") {  
  app.listen(PORT, () => {  
    console.log(`Server running on port ${PORT}`);  
  });  
}
```

```
    });  
  }  
  
  module.exports = app; // For testing
```

Now let's start dockerizing this application.

Phase 1: The Standard Baseline Approach

Like most developers, I started with something like this, a straightforward Dockerfile copied from a tutorial or Stack Overflow post, using `node:latest` as the base image without a second thought.

```
FROM node:latest  
  
WORKDIR /app  
  
COPY package*.json ./  
RUN npm install  
  
COPY . .  
  
EXPOSE 3000  
CMD ["node", "index.js"]
```

Building this image:

```
docker build -t node-app:phase1 .  
docker images | grep node-app
```

```
~/Documents/personal-projects/docker-shrink git:(main)±5 (0.126s)
docker images | grep node-app
node-app                                phase1                                9454273ba240    5 minutes ago    1.22GB
```

At 1.22 GB, this image is enormous. My application code might be just a few megabytes, but the image contained an entire Linux distribution, the Node.js runtime, and numerous dependencies I didn't need.

Testing our baseline image

Let's make sure our application works before proceeding with optimisations:

```
docker run -d -p 3000:3000 node-app:phase1
curl http://localhost:3000
```

```
~/Documents/personal-projects/docker-shrink git:(main)±5 (0.289s)
docker run -d -p 3000:3000 node-app:phase1
445b5b8bc2f908cbd8d5fdec86a239c7b06dcb1b1734f4f579c4c01dac796884

~/Documents/personal-projects/docker-shrink git:(main)±5 (0.102s)
curl http://localhost:3000
{"message":"Hello from a slim Docker container!","timestamp":"May 3rd 2025, 10:43:03 am","uptime":11.17499507}
```

Great! Our application is working correctly. Now let's start optimising.

Phase 2: Choosing the Right Base Image

My first optimisation was switching to a slimmer base image, which immediately brought several benefits:

- It eliminated unnecessary packages and utilities that weren't needed at runtime.
- It reduced the attack surface of the container.
- It significantly improved build and pull times during deployments.

Here's the updated Dockerfile:

```
FROM node:slim

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

EXPOSE 3000
CMD ["node", "index.js"]
```

Let's measure:

```
docker build -t node-app:phase2 . -f Dockerfile.phase2
docker images | grep node-app
```